

K-Means Clustering: 3 Lines of Code

From Chaos to Clusters in Seconds

Video Title: *K-Means Clustering: 3 Lines of Code to Group Your Data*

The Magic:

```
from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)
labels = kmeans.fit_predict(X)
```

Result: Data automatically grouped into 3 clusters!

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
import seaborn as sns

plt.style.use('dark_background')
np.random.seed(42)
```

1. The Problem: Ungrouped Data

```
In [2]: # Generate sample data – customer purchase behavior
X, y_true = make_blobs(n_samples=300, centers=3,
                        cluster_std=0.6, random_state=42)

# Visualize ungrouped data
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Before clustering
axes[0].scatter(X[:, 0], X[:, 1], c='gray', s=100,
                alpha=0.6, edgecolors='white', linewidth=1.5)
axes[0].set_xlabel('Feature 1 (e.g., Purchase Frequency)',
                  fontsize=13, fontweight='bold')
axes[0].set_ylabel('Feature 2 (e.g., Average Spend)',
                  fontsize=13, fontweight='bold')
axes[0].set_title('BEFORE: Ungrouped Data',
                  fontsize=16, fontweight='bold', color='red', pad=15)
axes[0].grid(alpha=0.3)

# Add question marks
axes[0].text(0.5, 0.95, 'Which customers are similar? 🤔',
            transform=axes[0].transAxes, ha='center', fontsize=12,
            color='yellow', fontweight='bold',
            bbox=dict(boxstyle='round', facecolor='black', alpha=0.8))
```

```

# After clustering preview
axes[1].scatter(X[:, 0], X[:, 1], c=y_true, s=100,
                cmap='viridis', alpha=0.6, edgecolors='white', linewidth=1.5)
axes[1].set_xlabel('Feature 1 (e.g., Purchase Frequency)',
                  fontsize=13, fontweight='bold')
axes[1].set_ylabel('Feature 2 (e.g., Average Spend)',
                  fontsize=13, fontweight='bold')
axes[1].set_title('AFTER: Clustered into 3 Groups',
                  fontsize=16, fontweight='bold', color='lime', pad=15)
axes[1].grid(alpha=0.3)

# Add checkmark
axes[1].text(0.5, 0.95, 'Clear customer segments! ✓',
            transform=axes[1].transAxes, ha='center', fontsize=12,
            color='lime', fontweight='bold',
            bbox=dict(boxstyle='round', facecolor='black', alpha=0.8))

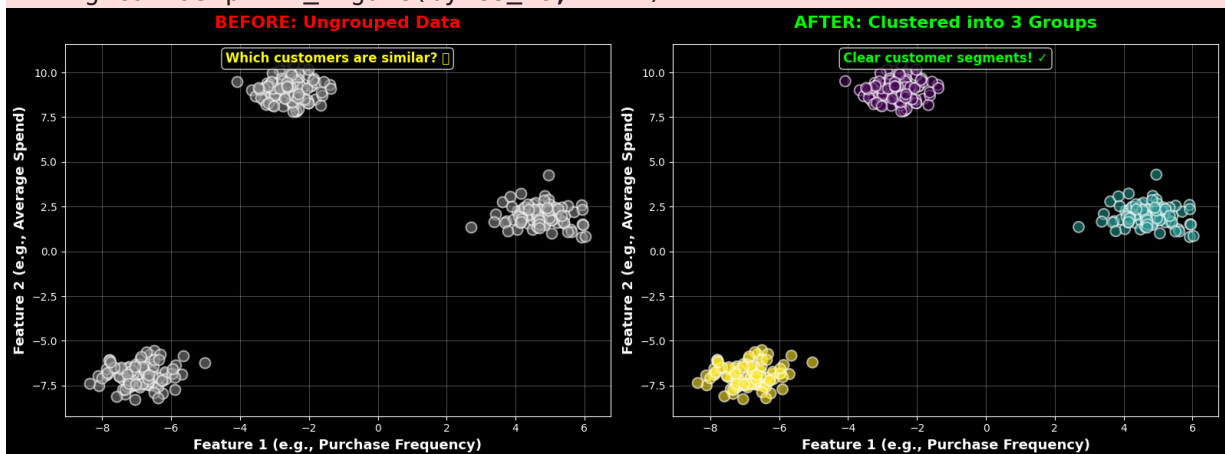
plt.tight_layout()
plt.show()

print("CLUSTERING TASK:")
print("="*70)
print(f"Data points: {len(X)}")
print(f"Features: {X.shape[1]}")
print("\nGOAL: Group similar data points together")
print("\nCHALLENGE: No labels provided (unsupervised learning!)")
print("\nSOLUTION: K-Means Clustering")
print("="*70)

```

/var/folders/62/mgtn51r97gl21dm8hd8ywt2h0000gq/T/ipykernel_8005/2781889948.py:42: UserWarning: Glyph 129300 (\N{THINKING FACE}) missing from font(s) DejaVu Sans.

```
plt.tight_layout()
/Users/durgesh.rathod/Documents/personal/youtube/youtube-tutorials-notebooks/.venv/lib/python3.12/site-packages/IPython/core/pylabtools.py:170: UserWarning: Glyph 129300 (\N{THINKING FACE}) missing from font(s) DejaVu Sans.
fig.canvas.print_figure(bytes_io, **kw)
```



CLUSTERING TASK:

=====

Data points: 300

Features: 2

GOAL: Group similar data points together

CHALLENGE: No labels provided (unsupervised learning!)

SOLUTION: K-Means Clustering

=====

2. The Magic: 3 Lines of Code!

```
In [3]: print("\n" + "="*70)
print("K-MEANS CLUSTERING IN 3 LINES:")
print("="*70 + "\n")

print("Line 1: Import")
print("-" * 40)
from sklearn.cluster import KMeans
print("from sklearn.cluster import KMeans")
print("✓ Import the algorithm\n")

print("Line 2: Create model")
print("-" * 40)
kmeans = KMeans(n_clusters=3, random_state=42)
print("kmeans = KMeans(n_clusters=3)")
print("✓ Specify number of clusters (3)\n")

print("Line 3: Fit and predict")
print("-" * 40)
labels = kmeans.fit_predict(X)
print("labels = kmeans.fit_predict(X)")
print("✓ Find clusters and assign labels\n")

print("="*70)
print("DONE! Data is now clustered! 🎉")
print("="*70)

# Get centroids
centroids = kmeans.cluster_centers_

print(f"\nRESULTS:")
print(f"Cluster 0: {np.sum(labels == 0)} points")
print(f"Cluster 1: {np.sum(labels == 1)} points")
print(f"Cluster 2: {np.sum(labels == 2)} points")
print(f"\nCentroids (cluster centers):")
for i, centroid in enumerate(centroids):
    print(f"Cluster {i}: ({centroid[0]:.2f}, {centroid[1]:.2f})")
```

K-MEANS CLUSTERING IN 3 LINES:

Line 1: Import

```
from sklearn.cluster import KMeans
✓ Import the algorithm
```

Line 2: Create model

```
kmeans = KMeans(n_clusters=3)
✓ Specify number of clusters (3)
```

Line 3: Fit and predict

```
labels = kmeans.fit_predict(X)
✓ Find clusters and assign labels
```

DONE! Data is now clustered! 🎉

RESULTS:

Cluster 0: 100 points
Cluster 1: 100 points
Cluster 2: 100 points

Centroids (cluster centers):

Cluster 0: (-2.58, 9.03)
Cluster 1: (-6.88, -6.94)
Cluster 2: (4.70, 2.00)

3. Visualizing the Result

```
In [4]: # Visualize clustered data
fig, ax = plt.subplots(figsize=(14, 10))

# Plot points colored by cluster
colors = ['cyan', 'magenta', 'lime']
cluster_names = ['Low Spenders', 'Medium Spenders', 'High Spenders']

for i in range(3):
    cluster_points = X[labels == i]
    ax.scatter(cluster_points[:, 0], cluster_points[:, 1],
               c=colors[i], s=150, alpha=0.6,
               edgecolors='white', linewidth=2,
               label=f'Cluster {i}: {cluster_names[i]}')

# Plot centroids
ax.scatter(centroids[:, 0], centroids[:, 1],
          c='red', s=500, marker='X',
          edgecolors='white', linewidth=3,
          label='Centroids (Centers)', zorder=5)
```

```

# Add centroid labels
for i, centroid in enumerate(centroids):
    ax.text(centroid[0], centroid[1] + 0.5, f'C{i}',
            ha='center', fontsize=14, fontweight='bold',
            color='white',
            bbox=dict(boxstyle='circle', facecolor='red',
                    edgecolor='white', linewidth=2))

# Draw cluster boundaries (Voronoi-like)
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200),
                    np.linspace(y_min, y_max, 200))
Z = kmeans.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

ax.contourf(xx, yy, Z, alpha=0.1, levels=2, colors=colors)
ax.contour(xx, yy, Z, alpha=0.3, levels=2, colors='white', linewidths=2)

ax.set_xlabel('Purchase Frequency', fontsize=14, fontweight='bold')
ax.set_ylabel('Average Spend ($)', fontsize=14, fontweight='bold')
ax.set_title('K-Means Clustering: Customer Segmentation',
            fontsize=18, fontweight='bold', color='cyan', pad=20)
ax.legend(fontsize=12, loc='upper right', framealpha=0.9)
ax.grid(alpha=0.3)

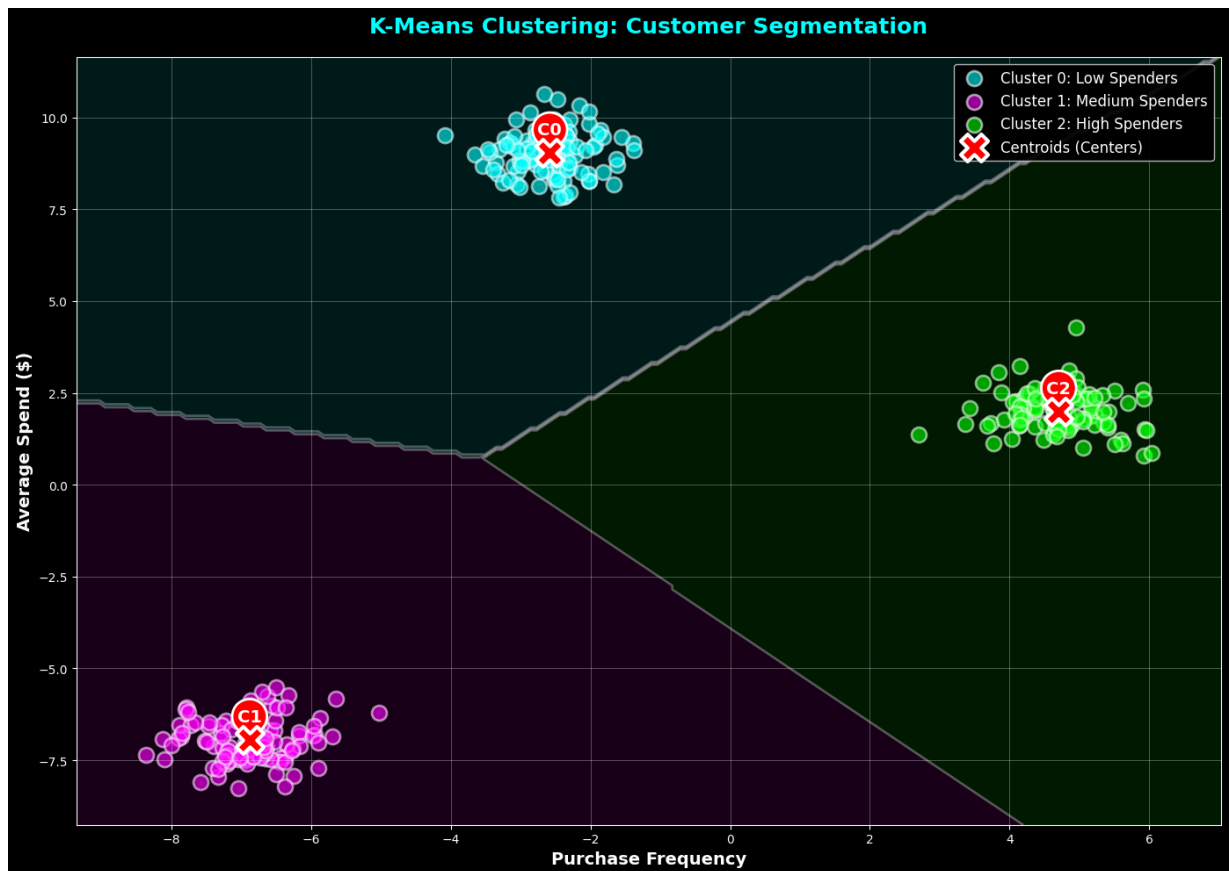
plt.tight_layout()
plt.show()

print("\nCLUSTER INTERPRETATION:")
print("="*70)
print("Cluster 0 (Cyan): Low Spenders")
print("  - Low purchase frequency")
print("  - Low average spend")
print("  - Strategy: Engagement campaigns\n")

print("Cluster 1 (Magenta): Medium Spenders")
print("  - Medium purchase frequency")
print("  - Medium average spend")
print("  - Strategy: Loyalty programs\n")

print("Cluster 2 (Lime): High Spenders")
print("  - High purchase frequency")
print("  - High average spend")
print("  - Strategy: VIP treatment")
print("="*70)

```



CLUSTER INTERPRETATION:

Cluster 0 (Cyan): Low Spenders

- Low purchase frequency
- Low average spend
- Strategy: Engagement campaigns

Cluster 1 (Magenta): Medium Spenders

- Medium purchase frequency
- Medium average spend
- Strategy: Loyalty programs

Cluster 2 (Lime): High Spenders

- High purchase frequency
- High average spend
- Strategy: VIP treatment

4. How K-Means Works: Step-by-Step

```
In [5]: # Demonstrate K-Means algorithm steps
np.random.seed(42)

# Simple 2D data for clear visualization
X_demo, _ = make_blobs(n_samples=50, centers=3, cluster_std=0.5, random_stat

# Manual K-Means steps
k = 3
```

```

# Step 1: Random initialization
initial_centroids = X_demo[np.random.choice(len(X_demo), k, replace=False)]

fig, axes = plt.subplots(2, 3, figsize=(18, 12))
axes = axes.ravel()

# Step 0: Initial data
axes[0].scatter(X_demo[:, 0], X_demo[:, 1], c='gray', s=100,
                alpha=0.6, edgecolors='white', linewidth=2)
axes[0].set_title('Step 0: Initial Data (Unlabeled)',
                  fontsize=14, fontweight='bold', color='yellow', pad=10)
axes[0].grid(alpha=0.3)

# Step 1: Random centroids
axes[1].scatter(X_demo[:, 0], X_demo[:, 1], c='gray', s=100,
                alpha=0.6, edgecolors='white', linewidth=2)
axes[1].scatter(initial_centroids[:, 0], initial_centroids[:, 1],
                c='red', s=500, marker='X',
                edgecolors='white', linewidth=3, zorder=5)
axes[1].set_title('Step 1: Initialize Random Centroids',
                  fontsize=14, fontweight='bold', color='yellow', pad=10)
axes[1].grid(alpha=0.3)

# Run actual K-Means and capture iterations
kmeans_demo = KMeans(n_clusters=3, init=initial_centroids,
                     n_init=1, max_iter=10, random_state=42)
kmeans_demo.fit(X_demo)

# Manually simulate iterations for visualization
centroids_iter = [initial_centroids.copy()]
labels_iter = []

for iteration in range(3):
    # Assign points to nearest centroid
    distances = np.sqrt(((X_demo[:, np.newaxis] - centroids_iter[-1])**2).sum(
        axis=1))
    labels = np.argmin(distances, axis=1)
    labels_iter.append(labels)

    # Update centroids
    new_centroids = np.array([X_demo[labels == i].mean(axis=0) for i in range(3)])
    centroids_iter.append(new_centroids)

    # Plot iteration
    ax = axes[2 + iteration]

    # Assign and update visualization
    for cluster_id in range(k):
        cluster_points = X_demo[labels == cluster_id]
        ax.scatter(cluster_points[:, 0], cluster_points[:, 1],
                  c=colors[cluster_id], s=100, alpha=0.6,
                  edgecolors='white', linewidth=2)

    # Old centroids (faded)
    ax.scatter(centroids_iter[iteration][:, 0], centroids_iter[iteration][:, 1],
              c='orange', s=300, marker='X', alpha=0.3,
              edgecolors='white', linewidth=2, label='Old')

```

```

# New centroids
ax.scatter(new_centroids[:, 0], new_centroids[:, 1],
           c='red', s=500, marker='X',
           edgecolors='white', linewidth=3, zorder=5, label='New')

# Arrows showing centroid movement
for i in range(k):
    ax.arrow(centroids_iter[iteration][i, 0], centroids_iter[iteration][i, 1],
             new_centroids[i, 0] - centroids_iter[iteration][i, 0],
             new_centroids[i, 1] - centroids_iter[iteration][i, 1],
             head_width=0.2, head_length=0.2, fc='yellow', ec='yellow',
             linewidth=2, alpha=0.7, zorder=4)

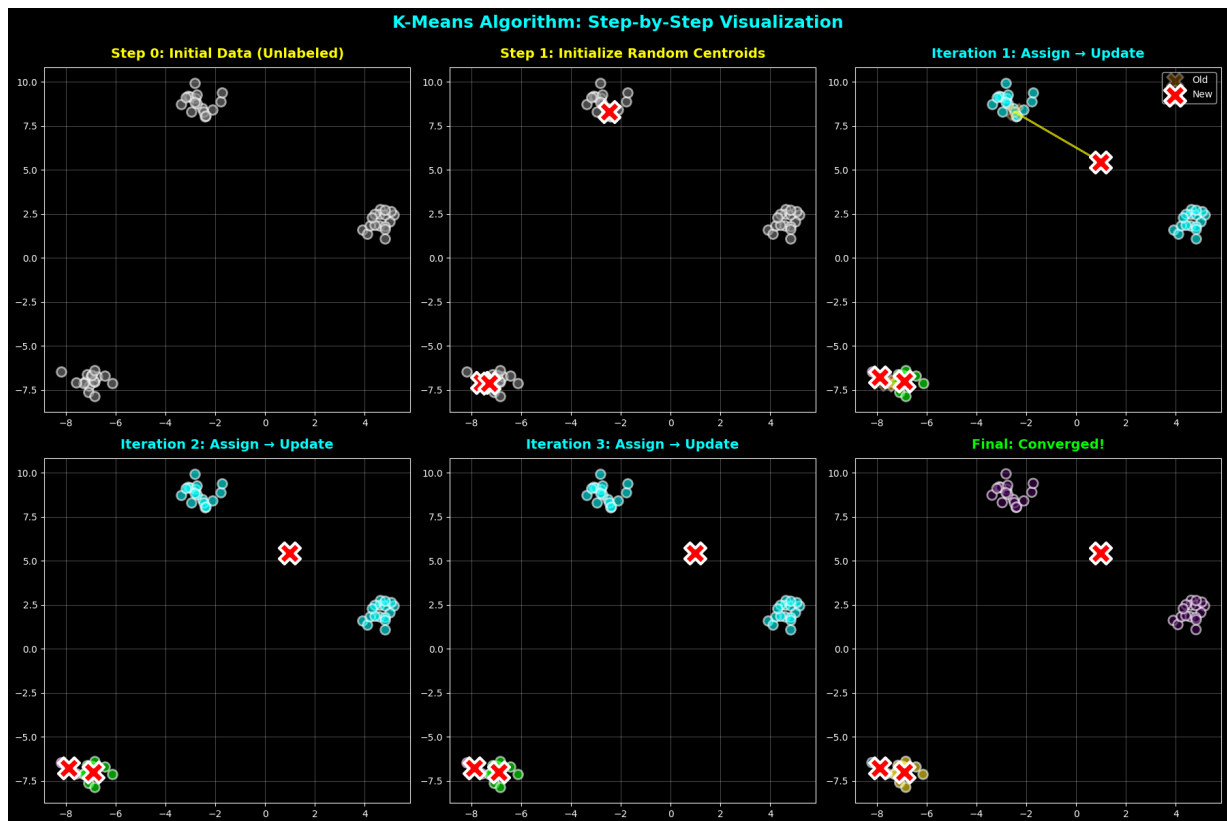
ax.set_title(f'Iteration {iteration + 1}: Assign → Update',
             fontsize=14, fontweight='bold', color='cyan', pad=10)
ax.grid(alpha=0.3)
if iteration == 0:
    ax.legend(fontsize=10, loc='upper right')

# Final result
axes[5].scatter(X_demo[:, 0], X_demo[:, 1], c=kmeans_demo.labels_,
                s=100, cmap='viridis', alpha=0.6,
                edgecolors='white', linewidth=2)
axes[5].scatter(kmeans_demo.cluster_centers_[:, 0],
                kmeans_demo.cluster_centers_[:, 1],
                c='red', s=500, marker='X',
                edgecolors='white', linewidth=3, zorder=5)
axes[5].set_title('Final: Converged!',
                  fontsize=14, fontweight='bold', color='lime', pad=10)
axes[5].grid(alpha=0.3)

plt.suptitle('K-Means Algorithm: Step-by-Step Visualization',
             fontsize=18, fontweight='bold', color='cyan', y=0.995)
plt.tight_layout()
plt.show()

print("\nK-MEANS ALGORITHM:")
print("="*70)
print("1. INITIALIZE: Pick K random centroids")
print("\n2. REPEAT until convergence:")
print("    a) ASSIGN: Each point → nearest centroid")
print("        Distance metric: Euclidean distance")
print("    b) UPDATE: Move centroids to cluster mean")
print("        New centroid = average of assigned points")
print("\n3. CONVERGE: When centroids stop moving")
print("    or max iterations reached")
print("\nTIME COMPLEXITY:  $O(n \times k \times i \times d)$ ")
print("    n = data points")
print("    k = clusters")
print("    i = iterations")
print("    d = dimensions")
print("="*70)

```



K-MEANS ALGORITHM:

1. INITIALIZE: Pick K random centroids
2. REPEAT until convergence:
 - a) ASSIGN: Each point → nearest centroid
Distance metric: Euclidean distance
 - b) UPDATE: Move centroids to cluster mean
New centroid = average of assigned points
3. CONVERGE: When centroids stop moving
or max iterations reached

TIME COMPLEXITY: $O(n \times k \times i \times d)$

n = data points

k = clusters

i = iterations

d = dimensions

5. Finding Optimal K: Elbow Method

```
In [6]: # Elbow method to find optimal number of clusters
inertias = []
K_range = range(1, 11)

for k in K_range:
    kmeans_temp = KMeans(n_clusters=k, random_state=42)
    kmeans_temp.fit(X)
    inertias.append(kmeans_temp.inertia_)
```

```

# Plot elbow curve
fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Elbow curve
axes[0].plot(K_range, inertias, 'o-', linewidth=3, markersize=10,
             color='cyan', markerfacecolor='magenta',
             markeredgecolor='white', markeredgewidth=2)

axes[0].set_xlabel('Number of Clusters (K)', fontsize=14, fontweight='bold')
axes[0].set_ylabel('Inertia (Within-Cluster Sum of Squares)',
                  fontsize=14, fontweight='bold')
axes[0].set_title('Elbow Method: Finding Optimal K',
                  fontsize=16, fontweight='bold', color='cyan', pad=15)
axes[0].grid(alpha=0.3)

# Highlight the elbow
elbow_k = 3
axes[0].scatter([elbow_k], [inertias[elbow_k-1]],
                s=500, c='lime', marker='*',
                edgecolors='white', linewidth=3, zorder=5)
axes[0].annotate('ELBOW!\nOptimal K=3',
                xy=(elbow_k, inertias[elbow_k-1]),
                xytext=(elbow_k+1.5, inertias[elbow_k-1]+20),
                arrowprops=dict(arrowstyle='->', color='lime', lw=3),
                fontsize=13, color='lime', fontweight='bold',
                bbox=dict(boxstyle='round', facecolor='black',
                        alpha=0.8, edgecolor='lime', linewidth=2))

# Comparison of different K values
k_values = [2, 3, 5]
k_colors_map = {2: 'red', 3: 'lime', 5: 'orange'}

for idx, k_val in enumerate(k_values):
    kmeans_k = KMeans(n_clusters=k_val, random_state=42)
    labels_k = kmeans_k.fit_predict(X)

    # Create subplot
    row = idx // 3
    col = idx % 3

    if idx == 0:
        ax = axes[1]
    else:
        continue # Only show K=2 for comparison

    ax.scatter(X[:, 0], X[:, 1], c=labels_k, s=100,
              cmap='viridis', alpha=0.6, edgecolors='white', linewidth=1.5)
    ax.scatter(kmeans_k.cluster_centers_[:, 0],
              kmeans_k.cluster_centers_[:, 1],
              c='red', s=400, marker='X',
              edgecolors='white', linewidth=3, zorder=5)

    status = 'x Too Few' if k_val == 2 else '✓ Optimal' if k_val == 3 else '
    color = k_colors_map[k_val]

```

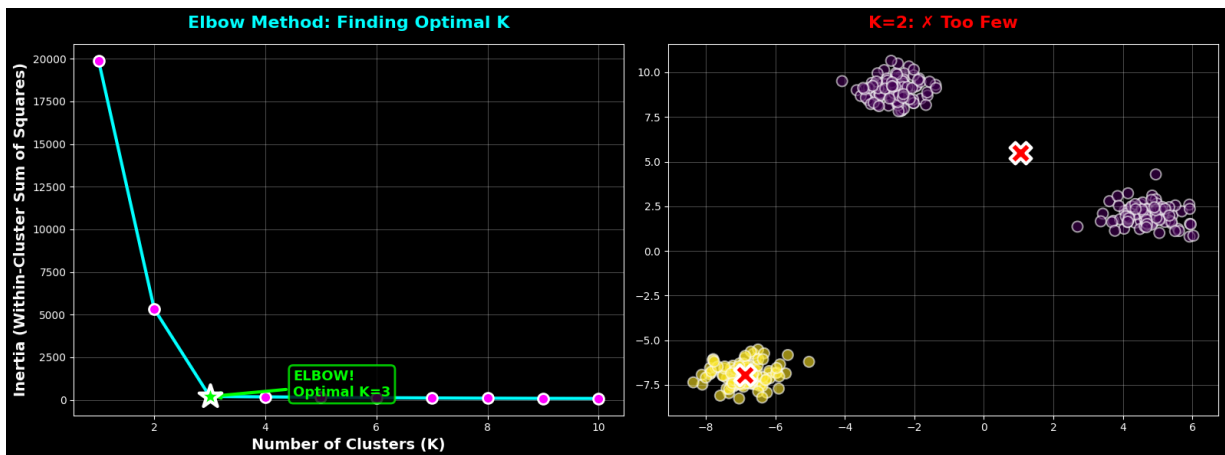
```

ax.set_title(f'K={k_val}: {status}',
             fontsize=16, fontweight='bold', color=color, pad=15)
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()

print("\nELBOW METHOD:")
print("="*70)
print("GOAL: Find optimal number of clusters (K)")
print("\nMETRIC: Inertia (within-cluster sum of squared distances)")
print("  - Lower inertia = tighter clusters")
print("  - But K=N gives inertia=0 (overfitting!)")
print("\nMETHOD:")
print("1. Try different K values (1 to 10)")
print("2. Calculate inertia for each K")
print("3. Plot K vs Inertia")
print("4. Find the 'elbow' (point of diminishing returns)")
print("\nINTERPRETATION:")
print("- Before elbow: Adding clusters helps a lot")
print("- At elbow: Optimal tradeoff")
print("- After elbow: Diminishing returns")
print("\nOur data: Elbow at K=3 ✓")
print("="*70)

```



ELBOW METHOD:

=====

GOAL: Find optimal number of clusters (K)

METRIC: Inertia (within-cluster sum of squared distances)

- Lower inertia = tighter clusters
- But K=N gives inertia=0 (overfitting!)

METHOD:

1. Try different K values (1 to 10)
2. Calculate inertia for each K
3. Plot K vs Inertia
4. Find the 'elbow' (point of diminishing returns)

INTERPRETATION:

- Before elbow: Adding clusters helps a lot
- At elbow: Optimal tradeoff
- After elbow: Diminishing returns

Our data: Elbow at K=3 ✓

=====

6. Real-World Example: Customer Segmentation

```
In [8]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Create realistic customer data
np.random.seed(42)

# 200 customers
n_customers = 200

# Generate features
# Cluster 0: Budget customers (low frequency, low spend)
freq_0 = np.random.normal(2, 0.5, 60) # ~2 purchases/month
spend_0 = np.random.normal(30, 5, 60) # ~$30/purchase

# Cluster 1: Regular customers (medium frequency, medium spend)
freq_1 = np.random.normal(8, 1, 80) # ~8 purchases/month
spend_1 = np.random.normal(70, 10, 80) # ~$70/purchase

# Cluster 2: VIP customers (high frequency, high spend)
freq_2 = np.random.normal(15, 2, 60) # ~15 purchases/month
spend_2 = np.random.normal(150, 20, 60) # ~$150/purchase

# Combine
purchase_frequency = np.concatenate([freq_0, freq_1, freq_2])
avg_spend = np.concatenate([spend_0, spend_1, spend_2])

X_customers = np.column_stack([purchase_frequency, avg_spend])

# Apply K-Means (THE 3 LINES!)
kmeans_customers = KMeans(n_clusters=3, random_state=42)
```

```

customer_labels = kmeans_customers.fit_predict(X_customers)

# Visualize
fig, ax = plt.subplots(figsize=(14, 10))

# Define segment names based on centroids
centroids_cust = kmeans_customers.cluster_centers_
segment_info = []

for i in range(3):
    freq = centroids_cust[i, 0]
    spend = centroids_cust[i, 1]

    if freq < 5:
        segment = 'Budget Shoppers'
        strategy = 'Engagement & Promotions'
        color = 'cyan'
    elif freq < 12:
        segment = 'Regular Customers'
        strategy = 'Loyalty Programs'
        color = 'magenta'
    else:
        segment = 'VIP Customers'
        strategy = 'Premium Services'
        color = 'lime'

    segment_info.append({'name': segment, 'strategy': strategy, 'color': color})

    cluster_points = X_customers[customer_labels == i]
    ax.scatter(cluster_points[:, 0], cluster_points[:, 1],
               c=color, s=150, alpha=0.6,
               edgecolors='white', linewidth=2,
               label=segment)

# Plot centroids (fixed marker)
ax.scatter(centroids_cust[:, 0], centroids_cust[:, 1],
          c='red', s=600, marker='P', # FIXED HERE
          edgecolors='white', linewidth=3,
          label='Segment Centers', zorder=5)

# Add segment labels
for i, info in enumerate(segment_info):
    ax.text(centroids_cust[i, 0], centroids_cust[i, 1] + 8,
           info['name'], ha='center', fontsize=12, fontweight='bold',
           color=info['color'],
           bbox=dict(boxstyle='round', facecolor='black',
                   alpha=0.9, edgecolor=info['color'], linewidth=2))

ax.set_xlabel('Purchase Frequency (per month)', fontsize=14, fontweight='bold')
ax.set_ylabel('Average Spend per Purchase ($)', fontsize=14, fontweight='bold')
ax.set_title('Customer Segmentation with K-Means\n3 Lines of Code, Actionable Insights',
            fontsize=18, fontweight='bold', color='cyan', pad=20)
ax.legend(fontsize=12, loc='upper left', framealpha=0.95)
ax.grid(alpha=0.3)

plt.tight_layout()

```

```

plt.show()

# Print results
print("\nCUSTOMER SEGMENTATION RESULTS:")
print("="*70)

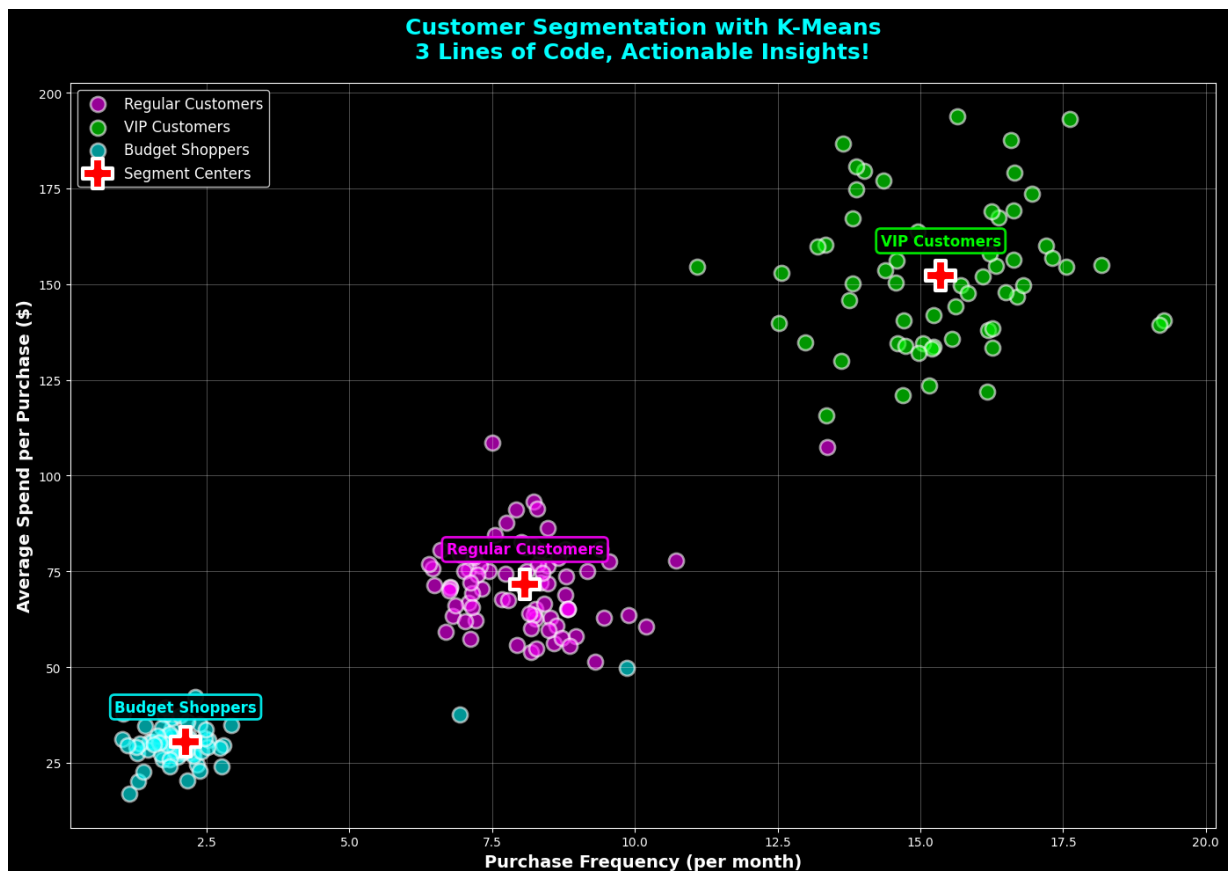
for i, info in enumerate(segment_info):
    n_customers_segment = np.sum(customer_labels == i)
    pct = (n_customers_segment / len(X_customers)) * 100

    print(f"\nSEGMENT {i}: {info['name']}")
    print(f"   Size: {n_customers_segment} customers ({pct:.1f}%)")
    print(f"   Center: Freq={centroids_cust[i, 0]:.1f}/month, Spend=${centroids_cust[i, 1]:.1f}")
    print(f"   Strategy: {info['strategy']}")

    # Calculate average lifetime value
    monthly_value = centroids_cust[i, 0] * centroids_cust[i, 1]
    yearly_value = monthly_value * 12
    print(f"   Est. Yearly Value: ${yearly_value:.2f}")

print("\n" + "="*70)
print("BUSINESS IMPACT:")
print("✓ Personalized marketing campaigns")
print("✓ Optimized resource allocation")
print("✓ Targeted promotions")
print("✓ Better customer retention")
print("="*70)

```



CUSTOMER SEGMENTATION RESULTS:

SEGMENT 0: Regular Customers

Size: 79 customers (39.5%)
Center: Freq=8.1/month, Spend=\$71.5
Strategy: Loyalty Programs
Est. Yearly Value: \$6932.97

SEGMENT 1: VIP Customers

Size: 59 customers (29.5%)
Center: Freq=15.4/month, Spend=\$152.1
Strategy: Premium Services
Est. Yearly Value: \$28040.31

SEGMENT 2: Budget Shoppers

Size: 62 customers (31.0%)
Center: Freq=2.1/month, Spend=\$30.4
Strategy: Engagement & Promotions
Est. Yearly Value: \$778.17

BUSINESS IMPACT:

- ✓ Personalized marketing campaigns
- ✓ Optimized resource allocation
- ✓ Targeted promotions
- ✓ Better customer retention

7. Key Takeaways

The 3 Lines:

```
from sklearn.cluster import KMeans # Line 1: Import
kmeans = KMeans(n_clusters=3)      # Line 2: Create
labels = kmeans.fit_predict(X)      # Line 3: Cluster!
```

How K-Means Works:

Algorithm:

1. **Initialize:** Pick K random centroids
2. **Assign:** Each point → nearest centroid (Euclidean distance)
3. **Update:** Move centroids to mean of assigned points
4. **Repeat:** Steps 2-3 until convergence

Convergence: When centroids stop moving or max iterations reached

Finding Optimal K:

Elbow Method:

- Plot K vs Inertia (within-cluster sum of squares)
- Find the "elbow" point
- Tradeoff: More clusters → lower inertia, but diminishing returns

Other Methods:

- Silhouette score (measures cluster separation)
- Gap statistic
- Domain knowledge (business requirements)

When to Use K-Means:

✓ Good for:

- Spherical clusters (similar sizes)
- Large datasets (fast algorithm)
- Known number of clusters
- Customer segmentation
- Image compression
- Document clustering

✗ Not good for:

- Non-spherical clusters (irregular shapes)
- Different cluster sizes
- Different cluster densities
- Noisy data with outliers

Advantages:

- ✓ Simple and intuitive
- ✓ Fast: $O(n \times k \times i \times d)$
- ✓ Scales well to large datasets
- ✓ Easy to implement (3 lines!)
- ✓ Guaranteed to converge

Disadvantages:

- ✗ Must specify K beforehand
- ✗ Sensitive to initialization (random start)
- ✗ Assumes spherical clusters
- ✗ Sensitive to outliers
- ✗ All features weighted equally (need scaling!)

Best Practices:

1. Scale Your Features:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
kmeans = KMeans(n_clusters=3)
labels = kmeans.fit_predict(X_scaled)
```

2. Run Multiple Times:

```
kmeans = KMeans(n_clusters=3, n_init=10) # 10 different
initializations
```

- Takes best result (lowest inertia)

3. Use Elbow Method:

- Don't guess K blindly
- Plot and visualize

4. Interpret Results:

- Look at cluster centers
- Understand business meaning
- Validate with domain experts

Real-World Applications:

1. Customer Segmentation:

- Group customers by behavior
- Personalized marketing
- Targeted promotions

2. Image Compression:

- Cluster pixel colors
- Reduce color palette
- Smaller file sizes

3. Anomaly Detection:

- Points far from any centroid = outliers
- Fraud detection
- System monitoring

4. Document Clustering:

- Group similar documents
- Topic modeling

- News categorization

5. Recommendation Systems:

- User clustering
- Product clustering
- Collaborative filtering

Remember:

- **Unsupervised learning** - no labels needed!
- **K must be specified** - use elbow method
- **Scale your features** - K-Means uses distance
- **Interpret results** - clusters should make business sense
- **3 lines of code** - but understanding matters! 🎯